

ROBOTICS PROJECT · RASPBERRY PI

INTELLIGENT CONTROL

Autonomous obstacle-avoidance robot

Complete technical file · design, assembly, commissioning

Python

OpenCV

RPi.GPIO

HC-SR04

L298N



INTRODUCTION

Overview and Objectives

The project involves building an **autonomous mobile robot** capable of detecting and avoiding obstacles in real time, without human intervention.

- 1 **Perceive** — measure the distance to obstacles and detect their position
- 2 **Decide** — choose the appropriate behavior based on the risk level
- 3 **Act** — drive both motors to avoid obstacles without stopping

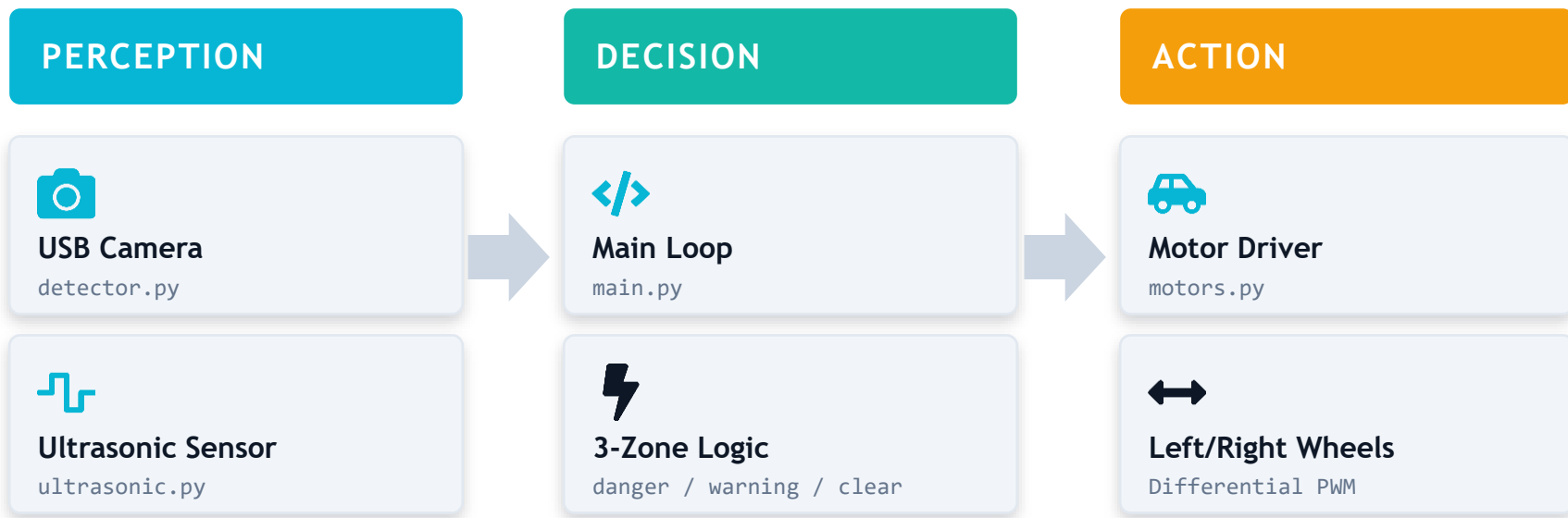
Constraint: all processing runs locally on the Raspberry Pi, with no GPU or network connection.



Overview of the finished robot

OVERVIEW

Software Architecture

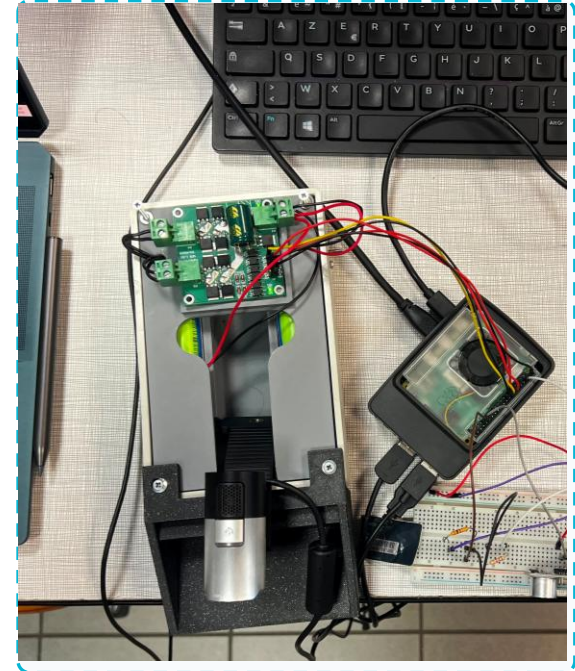


Real-time loop: perception → decision → action, running continuously. Four decoupled modules, each testable on its own.

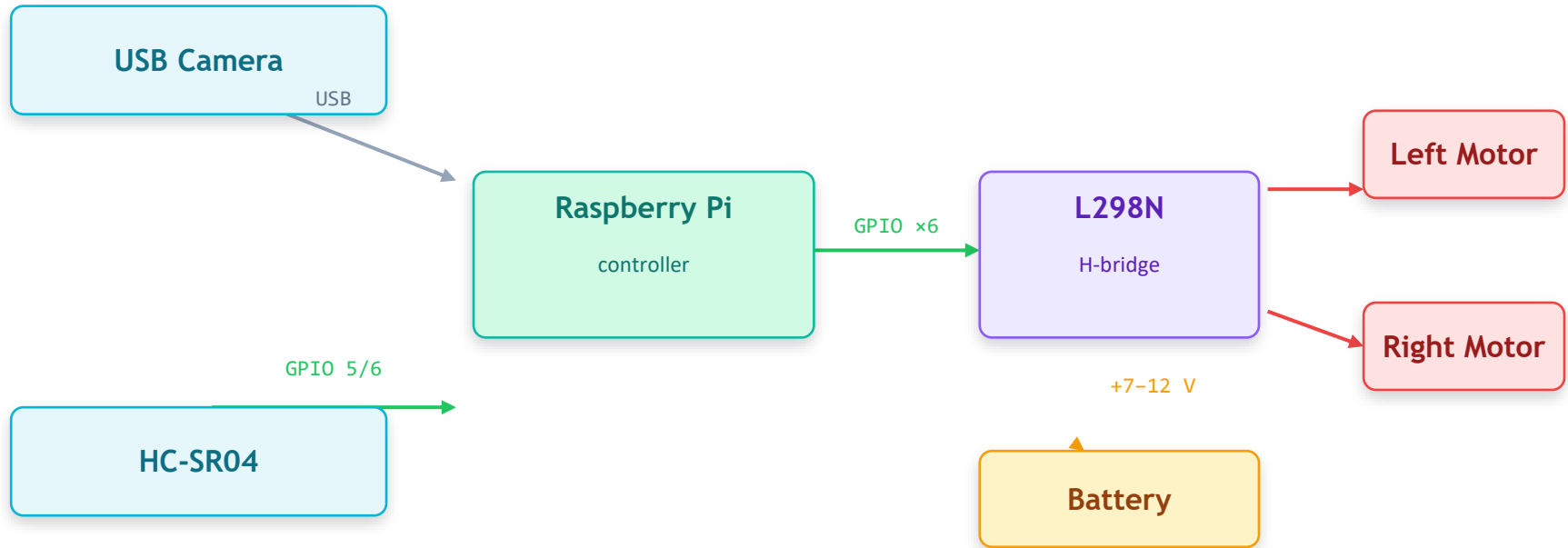
HARDWARE

Component List

Component	Role	Qty
Raspberry Pi 4/5	Brain: processing and control	1
HC-SR04	Distance sensor (ultrasonic)	1
USB camera	Lateral obstacle detection	1
Driver L298N	H-bridge: motor control	1
DC motors + wheels	Propulsion (left/right)	2
Battery 7-12 V	Motor power supply	1
R 1 k Ω + 2 k Ω	Voltage divider on the ECHO line	2
Chassis, wires, breadboard	Structure and wiring	—



Overall Wiring Diagram



Tip: connect all grounds together (Pi, L298N, battery) — this is the most common wiring mistake.

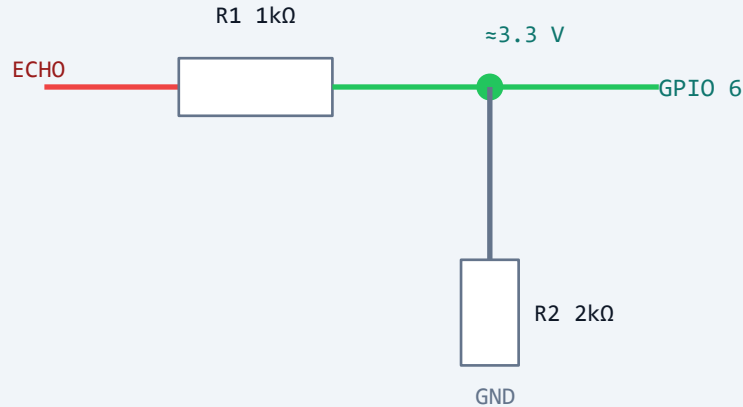
GPIO Pinout (BCM Numbering)

Pi Pin	Component	Function
GPIO 5	HC-SR04 TRIG	Triggers the ultrasonic pulse (direct connection)
GPIO 6	HC-SR04 ECHO	Echo return — via voltage divider
GPIO 17 / 27	L298N IN1 / IN2	Left motor direction
GPIO 22 / 23	L298N IN3 / IN4	Right motor direction
GPIO 18	L298N ENA	PWM speed — left motor
GPIO 24	L298N ENB	PWM speed — right motor
5 V / GND	HC-SR04 / L298N	Logic power and common ground

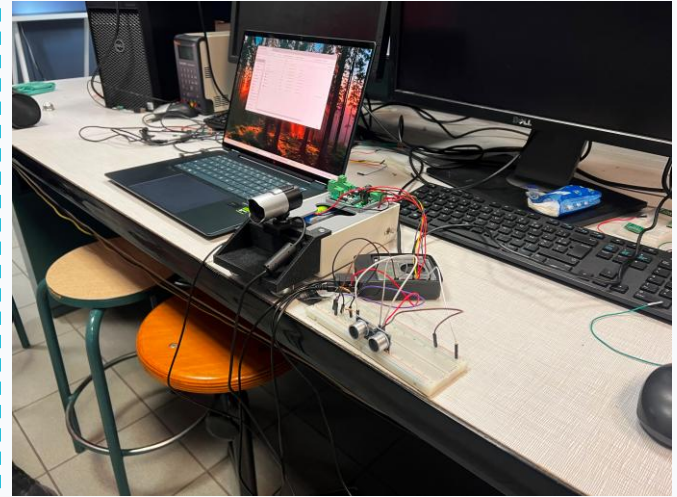
Only the ECHO line needs voltage adaptation: it outputs 5 V, while the Pi inputs tolerate only 3.3 V maximum.

Voltage Divider on the ECHO Line

Why: ECHO outputs a 5 V signal, incompatible with the Pi's 3.3 V inputs. Two resistors step down the voltage.



$$V = 5 \times R2 / (R1 + R2) = 5 \times 2/3 \approx 3.3 \text{ V}$$



Close-up of the voltage divider on the breadboard

The L298N Driver: a Dual H-Bridge

Four switches surround each motor. Closing one diagonal pair makes the current flow in one direction; the other diagonal reverses it.



Direction

The IN inputs set the direction: IN1=H, IN2=L → forward; reversed → backward.



Speed

A PWM signal on ENA/ENB sets the speed (duty cycle = % of speed).



Power

The motors are powered by the battery via the L298N, never by the Pi.

Good to know: the L298N heats up and “loses” 1.5–2 V; its protection diodes are already built in.

Time-of-Flight Distance Measurement

```
def distance_cm(self):
    GPIO.output(TRIG, GPIO.HIGH)
    time.sleep(0.00001) # 10 us
    GPIO.output(TRIG, GPIO.LOW)
    while GPIO.input(ECHO) == 0:
        start = time.time()
    while GPIO.input(ECHO) == 1:
        end = time.time()
    delta = end - start
    return delta * 34300 / 2 # cm
```

Principle

- ✓ 10 μ s TRIG pulse
- ✓ Measure the echo duration
- ✓ Distance = duration $\times$ 343 m/s $\div$ 2
- ✓ 30 ms timeout to prevent lock-up

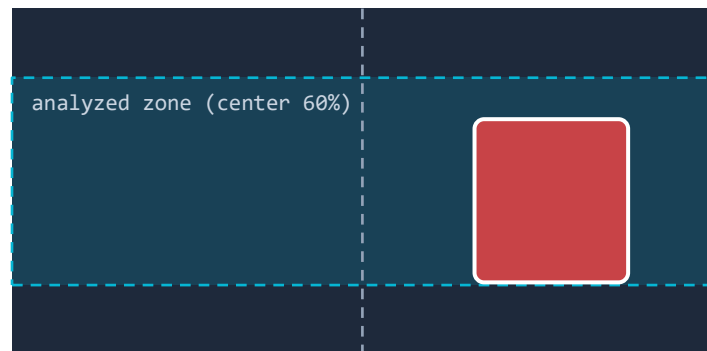
Range ~5 m · blind zone < 2 cm

Obstacle Detection (OpenCV)

```

# adaptive background subtraction (MOG2)
mask = self._bg.apply(band)
contours, _ = cv2.findContours(mask,
    cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
for cnt in contours:      # largest object
    if cv2.contourArea(cnt) < MIN_AREA:
        continue
    cx = int(M['m10'] / M['m00'])
# bias: -1 left, +1 right
bias = (cx - FRAME_W/2) / (FRAME_W/2)
```

Output: a lateral bias in [-1; +1]



The closest obstacle gives the direction to avoid;
combined with distance, it drives the steering.

Differential Control via L298N



```
def _set_motor(self, in1, in2, pwm, speed):  
    speed = max(-100, min(100, speed))  
    if speed >= 0:          # forward  
        GPIO.output(in1, GPIO.HIGH)  
        GPIO.output(in2, GPIO.LOW)  
    else:                   # reverse  
        GPIO.output(in1, GPIO.LOW)  
        GPIO.output(in2, GPIO.HIGH)  
    pwm.ChangeDutyCycle(abs(speed))
```

Sign of the speed → direction of rotation.
Absolute value → PWM duty cycle (the speed).

set(left, right) applies a speed to each wheel: it's the difference between the two that turns the robot.

Decision Based on Distance: 3 Zones



DANGER

distance < 15 cm

Stop, reverse for 0.5 s, then turn 180° away from the obstacle



WARNING

distance < 50 cm

Progressive avoidance: the camera gives the bias, proportional steering



CLEAR

distance ≥ 50 cm

Drive straight ahead at the base speed (60%)

"The closer the obstacle, the stronger the reaction."

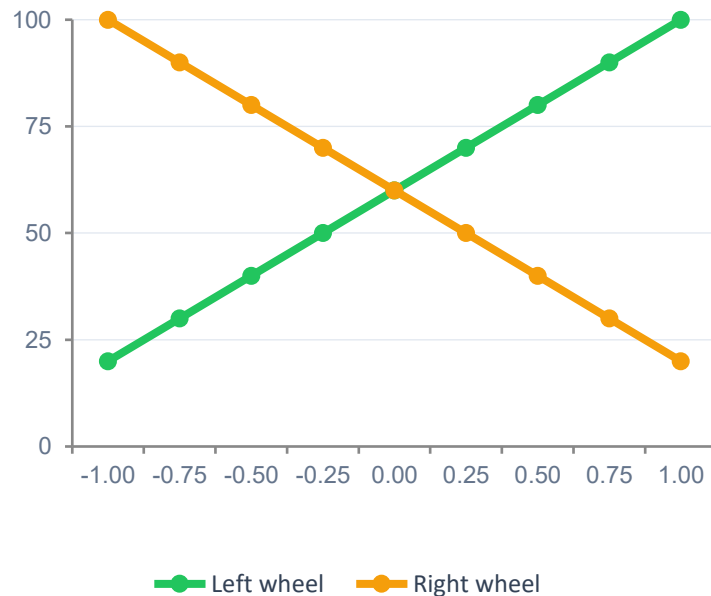
Proportional Steering



```
def compute_wheel_speeds(bias, distance):  
    proximity = (WARNING_CM - distance) / \  
                (WARNING_CM - DANGER_CM)  
    delta = bias * TURN_FACTOR * proximity  
    left = BASE_SPEED + delta  
    right = BASE_SPEED - delta  
    return left, right
```

delta combines **direction** (bias) and **urgency** (proximity). The wheels receive opposite speeds: the robot pivots toward the clear side.

Wheel Speed vs. Bias



COMMISSIONING

Testing Each Component Before the Loop

Before running `main.py`, test the sensor and motors separately. Wheels off the ground for the motor test.



`test_ultrasonic.py`

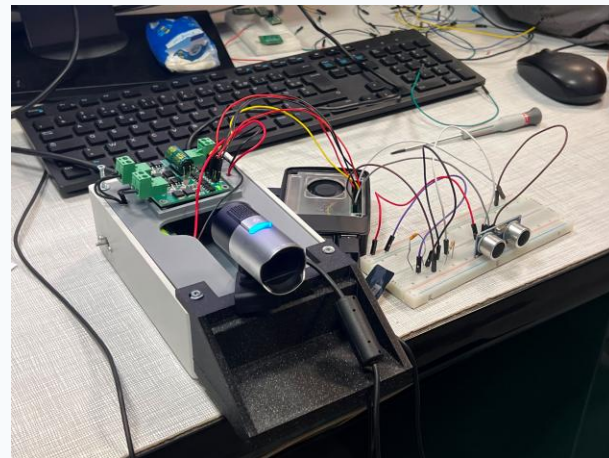
Continuously displays the distance: the value should follow your hand.



`test_motors.py`

Spins each motor forward/backward: check direction and side.

Direction reversed? Swap the motor's 2 output wires on the L298N — no code changes needed.



The robot propped up during testing

Running and Troubleshooting

Running



```
cd robot
python3 main.py

# clean stop: Ctrl + C
```

On a Pi with no screen (SSH), set `SHOW_VIDEO = False` in `main.py`.

Common Troubleshooting



« **SOC peripheral base address** » — RPi.GPIO incompatible (Pi 5 / Bookworm) → install `rpi-lgpio`



Sensor returns infinity — Check the voltage divider, the 5 V supply, and the common ground



Motor running backwards — Swap its 2 output wires on the L298N



Motors erratic — Missing common ground between Pi, L298N, and battery

CONCLUSION

Summary and Future Work

WHAT WORKS

- ✓ Fusion of ultrasonic (distance) + camera (direction)
- ✓ Reaction proportional to proximity
- ✓ Modular, testable architecture
- ✓ Real-time on Raspberry Pi, no GPU



POSSIBLE IMPROVEMENTS

- **PID control** — smooth the steering, avoid oscillations
- **ML-based detection** — embedded YOLO instead of background subtraction
- **Multiple sensors** — several HC-SR04 units to widen the field
- **Filtering** — averaging the sensor measurements



Space for a final photo of the robot in action